UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias
Escuela Profesional de Ciencia de la Computación

Curso: Fundamentos de Programación CC112 Semestre 2026-1

Laboratorio 3

Algoritmos de ordenamiento y búsqueda II

1. Sistema de Ranking Académico con Empates

Una facultad desea ordenar a sus estudiantes según sus calificaciones finales. Para ello, la información se encuentra almacenada en **dos arreglos paralelos** de tamaño n:

- `CODIGO[i]`: almacena el código del estudiante i.
- `NOTA[i]`: almacena la nota final del estudiante i.

Se requiere implementar un algoritmo basado en **Merge Sort** que ordene a los estudiantes bajo el siguiente criterio:

- Orden descendente según la nota.
- En caso de empate en la nota, el estudiante con menor código debe aparecer primero.

Importante: Dado que se trabaja con arreglos paralelos, cualquier intercambio o movimiento realizado durante el ordenamiento debe aplicarse simultáneamente en ambos arreglos para mantener la correspondencia entre código y nota.

Una vez ordenados los arreglos, se solicita implementar una función que, dado un código c, determine la posición en la que se encuentra dicho estudiante en el arreglo ordenado.

Restricción: Analice si es posible aplicar **búsqueda binaria** directamente sobre el arreglo ordenado. En caso contrario, justifique y utilice el método de búsqueda adecuado.

Entrada:

- $n = 5$
- `CODIGO = [101, 102, 103, 104, 105]`
- `NOTA = [15, 18, 18, 12, 15]`
- $c = 103$

Salida:

- Arreglos ordenados (mostrando código y nota).
- Posición del estudiante con código 103 en el arreglo ordenado.

2. **Detección de Errores en Sensores Industriales**

   Un sistema registra mediciones de temperatura cada minuto. Sin embargo, por fallas, algunas mediciones pueden ser inconsistentes.

   Se define una inconsistencia como un valor que rompe el orden creciente esperado del sistema.

   Diseñe un algoritmo eficiente usando **divide y vencerás** para detectar cuántas inconsistencias existen (similar al conteo de inversiones).

   Entrada: $n = 6$ $[10, 12, 15, 11, 18, 17]$

   Salida: Número de inconsistencias: 3

3. **Búsqueda en Base de Datos con Rango de Valores**

   Una base de datos contiene valores ordenados de presión atmosférica.

   Dado un rango $[L, R]$, determine cuántos valores pertenecen a ese rango utilizando únicamente **búsqueda binaria**.

   Entrada: $n = 7$ $[5, 8, 10, 12, 15, 18, 20]$ $L = 9,\ R = 15$

   Salida: Cantidad: 3

4. **Optimización de Cargas en un Servidor**

   Un servidor procesa tareas con distintos tiempos de ejecución. Se desea dividir el arreglo en dos partes de tal forma que la diferencia entre la suma de ambas partes sea mínima.

   Proponga una solución usando **divide y vencerás**.

   Entrada: $n = 6$ $[3, 1, 4, 2, 2, 1]$

   Salida: Diferencia mínima posible

5. **Determinación de Pico en Señales**

   Una señal digital está representada por un arreglo que primero crece y luego decrece (forma de montaña).

   Implemente un algoritmo basado en **búsqueda binaria** para encontrar el valor máximo.

   Entrada: $n = 7$ $[1, 3, 8, 12, 9, 5, 2]$

   Salida: Valor máximo: 12

6. **Planificación de Entregas Urgentes**

   Una empresa logística tiene paquetes con tiempos límite. Se desea ordenar los paquetes de menor a mayor tiempo límite usando **Quick Sort**.

   Posteriormente, determinar si existe algún paquete cuyo tiempo límite sea exactamente igual a un valor crítico T usando **búsqueda binaria**.

   Entrada: $n = 5$ $[30, 10, 20, 50, 40]$ $T = 20$

   Salida: Lista ordenada Existe / No existe

7. **Análisis de Transacciones Financieras**

   Dado un arreglo de transacciones (ganancias y pérdidas), determine el subarreglo contiguo con la mayor suma utilizando **divide y vencerás**.

   Entrada: $n = 8$ $[-2, -3, 4, -1, -2, 1, 5, -3]$

   Salida: Máxima ganancia: 7

8. **Selección de Candidatos Prioritarios**

   Una lista de candidatos tiene puntajes distintos. Se desea encontrar el **k-ésimo mejor candidato** sin ordenar completamente el arreglo.

   Utilice una estrategia basada en **Quickselect**.

   Entrada: $n = 7$ $[70, 85, 60, 90, 75, 80, 65]$ $k = 3$

   Salida: Tercer mejor puntaje

9. **Comparación de Listas de Inventario**

   Dos almacenes tienen inventarios ordenados. Determine si existe algún elemento en común entre ambos usando **búsqueda binaria**.

   Entrada: $A = [1, 3, 5, 7, 9]$ $B = [2, 4, 5, 8, 10]$

   Salida: Existe elemento común

10. **Reconstrucción de Orden Original**

    Se tiene un arreglo que fue ordenado, pero se conoce además un arreglo auxiliar que indica cuántos elementos menores había antes de cada posición original.

    Reconstruya el arreglo original utilizando técnicas de **divide y vencerás**.

    Entrada: Valores: $[10, 20, 30, 40]$ Posiciones relativas: $[0, 1, 1, 3]$

    Salida: Arreglo original reconstruido

11. **Verificación de Frases Palíndromas con Recursión**

    Una frase se considera palíndroma si, al invertirla, resulta igual a la original, ignorando los espacios en blanco, las mayúsculas/minúsculas y los acentos.

    Por ejemplo, las siguientes frases son palíndromas:

    - "dabale arroz a la zorra el abad"
    - "a ti no bonita"
    - "animo Romina"
    - "La ruta nos aporto otro paso natural"
    - "Se verla al reves"

    Utilizando **recursión**, escriba un programa en C++ que determine si una frase ingresada es o no un palíndromo bajo las condiciones descritas.

    Entrada: Una línea de texto que representa una frase.

    Salida: Indicar si la frase es palíndroma o no.

    Sugerencias:

    - Para comprobar si un carácter es un espacio, utilice la función `isspace()`.
    - Para convertir caracteres a minúsculas, utilice la función `tolower()`.

12. **Optimización de Compra en Subasta de Panetones**

    A inicios de enero, en una subasta de mercadería decomisada por la SUNAT, se ofrece un lote de N panetones.

El precio de cada panetón se almacena en un arreglo `PRECIO`, mientras que el mes de la fecha de vencimiento se almacena en un arreglo `MES`. Considere que todos los panetones vencen dentro del mismo año y que el precio tiende a ser menor cuando la fecha de vencimiento es más cercana.

Por razones comerciales, **no se deben comprar panetones que venzan en los meses de marzo (3) y abril (4)**, ya que en dichos meses la demanda es muy baja.

El objetivo es seleccionar exactamente K panetones válidos de tal manera que el costo total de compra sea mínimo.

Entrada:

- Un entero N que representa la cantidad de panetones.
- Un arreglo `PRECIO` de tamaño N.
- Un arreglo `MES` de tamaño N.
- Un entero K que indica la cantidad de panetones a comprar.

Salida:

- El costo mínimo total por la compra de K panetones que cumplan las restricciones.

Ejemplo:

- `PRECIO = [60, 55, 30, 25, 45, 15, 30, 20, 50, 45]`
- `MES = [10, 9, 4, 3, 7, 1, 4, 2, 8, 7]`

Entonces el programa puede mostrar las siguientes salidas:

```
Numero de panetones a comprar: 11
Solo se pudieron comprar 10 panetones.
Monto gastado en la compra: 290 soles.
--------------------------------
Process exited after 62.04 seconds with return value 0
Presione una tecla para continuar . . .
```

```
Numero de panetones a comprar: 5
Monto gastado en la compra: 175 soles.
--------------------------------
Process exited after 3.16 seconds with return value 0
Presione una tecla para continuar . . .
```

13. Años Bisiestos y Búsqueda Lineal

Son años bisiestos todos aquellos que son múltiplos de 4, excepto los que son múltiplos de 100 pero no de 400.

Implemente un programa que reciba por teclado dos años: N (año inicial) y M (año final), y que almacene en un arreglo todos los años bisiestos comprendidos entre ambos valores.

El programa debe realizar lo siguiente:

- Calcular primero la cantidad de años bisiestos en el intervalo.

- Almacenar dichos años en un arreglo.
- Mostrar la lista de años bisiestos encontrados.
- Solicitar un año adicional.
- Utilizando **búsqueda lineal**, determinar el año bisiesto inmediato inferior al año ingresado.
- Si el año ingresado no pertenece al rango considerado, mostrar el mensaje: ‘‘`año fuera de rango`’’.

Ejemplos de ejecución:

Caso 1

```
Ingrese año inicial: 2000
Ingrese año final: 2100
La lista de bisiestos entre 2000 y 2100 es:
2000 2004 2008 2012 2016 2020 2024 2028 2032 2036 2040 2044 2048 2052
2056 2060 2064 2068 2072 2076 2080 2084 2088 2092 2096
Ingrese un año a buscar: 2018
El año bisiesto inmediato inferior es: 2016
```

Caso 2

```
Ingrese año inicial: 1900
Ingrese año final: 2000
La lista de bisiestos entre 1900 y 2000 es:
1904 1908 1912 1916 1920 1924 1928 1932 1936 1940 1944 1948 1952 1956
1960 1964 1968 1972 1976 1980 1984 1988 1992 1996 2000
Ingrese un año a buscar: 1924
El año bisiesto inmediato inferior es: 1920
```

Caso 3

```
Ingrese año inicial: 1800
Ingrese año final: 1900
La lista de bisiestos entre 1800 y 1900 es:
1804 1808 1812 1816 1820 1824 1828 1832 1836 1840 1844 1848 1852 1856
1860 1864 1868 1872 1876 1880 1884 1888 1892 1896
Ingrese un año a buscar: 2010
año fuera de rango
```

14. Búsqueda en Arreglo Rotado (Divide y Vencerás)

Dado un arreglo de enteros A de tamaño n, ordenado de manera ascendente y cuyos elementos son todos distintos, se requiere implementar las siguientes funciones:

- **(a) Rotación del arreglo**
  Implemente una función que, dado un entero k tal que $1 \leq k < n$, desplace los elementos del arreglo de manera circular hacia la izquierda, generando un nuevo arreglo de la forma:

$$[A[k], A[k+1], \ldots, A[n-1], A[0], A[1], \ldots, A[k-1]]$$

Ejemplo:

Si $A = [0, 1, 2, 3, 4, 5, 6, 7]$ y $k = 3$, el arreglo resultante es:

$$[3, 4, 5, 6, 7, 0, 1, 2]$$

- **(b) Búsqueda eficiente en arreglo rotado**

  Implemente una función que reciba como argumentos el arreglo rotado, su tamaño n y un entero c, y retorne el índice en el que se encuentra c dentro del arreglo. Si el elemento no existe, la función debe retornar -1.

  La implementación debe tener complejidad temporal $\mathcal{O}(\log n)$, por lo que **no se permite el uso de búsqueda lineal**.

  Ejemplos:

  - Input: $[4, 5, 6, 7, 0, 1, 2]$, $c = 0$ Output: 4
  - Input: $[4, 5, 6, 7, 0, 1, 2]$, $c = 8$ Output: -1

Nota: Un algoritmo tiene complejidad logarítmica cuando en cada paso reduce significativamente el tamaño del problema (por ejemplo, a la mitad), como ocurre en la búsqueda binaria. Este tipo de algoritmos es altamente eficiente y fundamental en el diseño de soluciones mediante **divide y vencerás**.

15. Escriba un programa que calcule en forma recursiva una matriz $A_{n \times n}$. Los elementos $a_{i,j}$ sobre la diagonal de A son nulos y los elementos restantes se forman de la suma $a_{i-1,j-1} + a_{i-1,j}$. Ejemplo de salida

```
1   0   0   0   0   0   0
1   1   0   0   0   0   0
1   2   1   0   0   0   0
1   3   3   1   0   0   0
1   4   6   4   1   0   0
1   5  10  10   5   1   0
1   6  15  20  15   6   1
```